

Very High speed Integrated Circuit (VHSIC) Hardware Description Language

Use of VHDL :- To design digital systems before building them

- To simulate & test circuits

- To reduce error and costs

General structure of VHDL program

VHDL is made up of different blocks, main three parts: 1. Library section

2. Entity Declaration

3. Architecture Declaration

1. Library section

- Provides predefined data types and functions. (e.g. std-logic, std-logic, vector)

e.g. library ieee;

use ieee.std-logic-1164.all;

2. Entity section

- Define external view (black box) of system, specifies inputs and outputs

Syntax:

```
entity entity-name is
  port (
    signal-name : mode type
  );
```

Modes:

in

out

inout

buffer

3. Architecture section

- connected to entity, defines how system work internally.

- Can be written in different forms: Behavioral, Dataflow, structural

- Two parts:

1. Declaration part

Signals, constants, components

2. Execution part

Logic statements

architecture arch-name of entity-name is

-- declarations

begin

-- logic implementation (execution part)

end arch-name;

Optional parts:

4. Package

Package is a VHDL design unit used to declare common data types, constants, functions and procedures that can be shared across multiple VHDL modules.

- Reusability, used with "use" statement.

For e.g. use IEEE.std-logic-1164.all;

→ Package

(i) ~~Package declaration~~

package my-package is

constant N : integer := 8;

end my-package;

(ii) ~~Package body~~

library ieee;

use ieee.std-logic-1164.all;

use work.my-package.all;

std-logic-vector (N downto 0).

5. Configuration

- Used to bind an entity to an architecture.

- Used when multiple architectures exist for one entity.

Modeling styles

1. Behavioral Modeling

- Describes what system does

, Highest Level Abstraction

- Uses process block

- Written like a program (sequential statements)

- Simulation oriented allowing designers to validate functionality before moving to hardware implementation.

Concurrent statement
→ statement running parallelly.

Sequential statements
→ statement execute one after another

Combinational ckt → No memory
Sequential ckt → Memory used.

Date _____
Page _____

2. Dataflow Modeling

- Describes how data flows from i/p to o/p, Middle Level Abstraction.
- Uses concurrent statements.
- Based on boolean expressions, logical equations, simple relationships betⁿ i/p & o/p.

3. Structural Modeling

- Describes how system is built using components (gates/modules), lowest level abstraction
- Focus : structure (hardware connection), closely reflects actual hardware, suitable for managing large & intricate designs
- Similar to circuit diagram, Elements: component declaration, component instantiation, Port Mapping

Datatypes in VHDL

Datatypes define what kind of values a signal, variable or constant can hold.

- Two main categories:
1. Scalar Type - single value
 2. Composite Type - multiple value combined

1. Scalar Type

a) predefined scalar type: built-in provided by VHDL

Eg. bit → '0', '1'

boolean → TRUE, FALSE

integer → whole numbers

real → decimal numbers

std-logic → multi-valued logic

e.g. signal a: std-logic;
signal count: integer;

b) User defined data type : User defines list of name values.

Used in Finite State Machines, Control system

e.g. type state is (IDLE, START, STOP);

signal s: state

s can take only one of these values.

2. Composite type → store multiple values.

a) Array Type : Collection of same type element

Used for data buses, memory.

Eg. bit-vector

std-logic-vector

signal data : std-logic-vector (7 downto 0);

b) Record Type : collection of different data types

Used for complex data structures.

Eg. type student is record

name : string (1 to 10);

age : integer;

end record;

Subprogram

Subprograms are reusable blocks of code used to perform specific task in VHDL design.

Benefits : Code readability, Modularity, reusability, Maintainability

Types :

1. Function

- Takes input, perform computation & return value.

- Must return one value.

- cannot modify signal directly.

e.g. function add (a,b : integer) return integer is

begin

return a+b;

end function;

Use : signal result : integer;
result <= add (5,3);

2 Procedure

- can take multiple input/output, don't return value directly
- can modify signal/variables, uses in, out, inout parameters

```
procedure add-proc1  
    signal a,b : in integer;  
    signal result : out integer  
    ) is  
begin  
    result <= a+b;  
end procedure;
```

Use: add-proc1(x,y,sum);

Structural Elements:

1. Component Declaration

Defines interface of component by specifying its input & output ports. Component declarations provide a blueprint for components to be used & are placed in architecture section or package.

component componentName is

Port /

Port 1 : in STD-LOGIC;

Port 2 : in std-logic;

-- more ports)

);

end component;

pn

2. Component Instantiation & port mapping

- Instantiation is used to create physical representations in design. Each instance correspond to individual realization.
- Port mapping connects signal defined in architecture to port of components during instantiation.
- It ensures proper flow of data betⁿ components.

syn^{tax}: InstanceLabel : componentName

Port map (

port 1 $\Rightarrow$ signal 1,

port 2 $\Rightarrow$ signal 2

-- Additional mapping

);

4. Mixed Modeling

Combines 3 modeling styles. Allows designers to leverage strength of each style for diff. part of system. Highly practical method $\rightarrow$ flexibility, readability, scalability

Package

Package is a VHDL design unit used to declare common data types, constants, functions, procedures that can be shared across multiple VHDL modules.

Structure of package:

1. Package declaration
2. Package Body

1. Package declaration ; contains type definitions, constants, function/procedure declaration

package my_package is

constant N : integer := 8;

function add (a, b : integer) return integer;

end my_package;

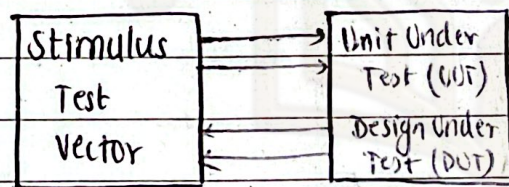
2. Package body : contains actual implementation of functions/procedure

```
package body my-package is
    function add (a,b : integer) return integer is
    begin
        return a+b;
    end function;
end my-package;
```

Using: library IEEE;
use work.my-package.all;

Testbench

Testbench is a VHDL program used to verify the functionality of a design (VUT/DUT) by applying different input conditions and observing the output response.



Testbench Architecture

Components: 1. Entity Declaration

2. Architecture Declaration

3. Component Declaration

4. Signal Declaration

5. Clock and stimulus Generation

6. component Instantiation

Behavioral $\Rightarrow$ Process

if, elsif, else

if (condition)

end if;

Date _____
Page _____

Universal gates : NAND, NOR

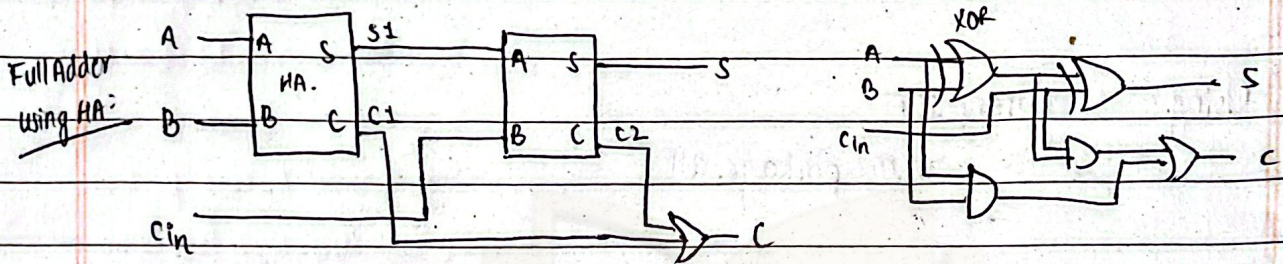
Special gates : XOR, XNOR

XOR			}	XNOR
A	B	Output		Output
0	0	0		1
0	1	1		0
1	0	1		0
1	1	0		1

Adder

$$\text{Sum} = A \text{ xor } B \quad \left\{ \begin{array}{l} A \text{ xor } B \text{ xor } C \\ (A \text{ and } B) \text{ or } (B \text{ and } C) \text{ or } (A \text{ and } C) \end{array} \right.$$

$$\text{Carry} = A \text{ and } B$$



code for full adder using half adder as component:

VHDL code for HA:

```
library IEEE;  
use IEEE.std_logic_1164.all;
```

```
entity half-adder is  
  port (  
    A, B : in std_logic;  
    sum, carry : out std_logic  
  );  
end half-adder
```

```
architecture behavioral of half-adder is  
  begin  
    sum <= A xor B;  
    carry <= A and B;  
  end behavioral;
```

VHDL code for ORGate:

```
library IEEE;
use IEEE.std-logic-1164.all;
```

```
entity ORGate is
port (
    X, Y : in std-logic;
    Z : out std-logic);
end ORGate;
```

```
architecture behavioral of ORGate is
begin
    Z <= X or Y;
end behavioral;
```

FA:

```
library IEEE;
use IEEE.std-logic-1164.all;
```

```
entity FullAdder is
port (A, B, Cin : in std-logic;
    sum, carry : out std-logic);
end FullAdder;
```

architecture structural of FullAdder is

```
component HalfAdder
port (
    A, B : in std-logic;
    sum, carry : out std-logic);
end component;
```

```
component ORGate
port (X, Y : in std-logic;
    Z : out std-logic);
end component;

signal s1, c1, c2 : std-logic;

begin
    HA1 : HalfAdder port map
        (A => A, B => B, sum => s1, carry => c1);

    HA2 : HalfAdder port map
        (A => s1, B => Cin, sum => sum, carry => c2);

    OR1 : ORGate port map
        (X => c1, Y => c2, Z => carry);
end structural;
```

Testbench for Full Subtractor:

Subtractor

$$\begin{aligned} \text{diff} &\leq A \text{ xor } B & \left\{ \begin{aligned} \text{diff} &\leq A \text{ xor } B \text{ xor } C \\ \text{borrow} &\leq (\text{not } A \text{ and } B) \text{ or } (\text{not } A \text{ and } C) \text{ or } (B \text{ and } C) \end{aligned} \right. \\ \text{borrow} &\leq (\text{not } A) \text{ and } B \end{aligned}$$

Testbench for FS:

```
library IEEE;
use IEEE.std_logic_1164.all;
```

A	B	Bin	diff	borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

```
entity test_full_sub is
    end end test_full_sub;
```

architecture arch_fs of test_full_sub is

component full_sub

port (A, B, Bin : in std_logic;
diff, borrow : out std_logic);

end component;

signal A, B, Bin, ~~diff, borrow~~ : std_logic := '0';

signal diff, borrow : std_logic;

begin

UUT : full_sub port map

(A => A, B => B, Bin => Bin, diff => diff, borrow => borrow);

stim_proc : process

begin

A <= '0'; B <= '0'; Bin <= '0'; wait for 100 ps; ---

end process;

end arch_fs;

case:

when " " => --

When others =>
end case;

Date _____
Page _____

Multiplexer

<u>4:1</u>	S1	S0	Y
	0	0	I(0)
	0	1	I(1)
	1	0	I(2)
	1	1	I(3)

entity mux is

port (I : in std-logic-vector (3 downto 0); (Input)

S : in std-logic-vector (1 downto 0); (Select / Input)

Y : out std-logic-1; (Output)

end mux;

process(I,S) begin

case S is

when "00" => Y <= I(0);

end case; others => Y <= 0;

Y <= (not S1 and not S0 and I(0))

or (not S1 and S0 and I(1)) or

Test bench -> Initialize value for I signal signal I : std-logic-vector (3 downto 0) := "0101";

De-Multiplexer

<u>1:4</u>	S1	S0	Active o/p
	0	0	Y(0)
	0	1	Y(1)
	1	0	Y(2)
	1	1	Y(3)

entity demux is

port (I : in std-logic;

S : in std-logic-vector (1 downto 0);

Y : out std-logic-vector (3 downto 0));

end demux;

Y(0) <= I and not S1 and not S0

Y(1) <= I and not S1 and S0

case S is

when "00" => Y(0) <= I;

others => Y(0) <= "0000".

Initialize I <= 1 inside process (stim_proc: process).

Encoder

	D	A
0 0	0 0 0 0 0 1	0 0 0
0 0	0 0 0 0 1 0	0 0 1
0 0	0 0 0 1 0 0	0 1 0

port (D : in std-logic-vector (7 downto 0);

A : out std-logic-vector (2 downto 0));

A(0) <= D(1) or

A(1) <= D(2) or

when "00000001" => A <= "000";

Decoder

I	Y
000	0 0 0 0 0 0 0 1
001	0 0 0 0 0 0 1 0

I : in std-logic-vector (2 downto 0);

Y : out std-logic-vector (7 downto 0));

case I is

when "000" => Y <= "00000001";

Y(0) <= not I(0) and not I(1) and not I(2)

Flipflops

S	R	Q(Next state)	J	K	Q(Next State)	D	Q(Next state)
0	0	No change	0	0	No change	0	0
0	1	Reset (0)	0	1	Reset (0)	1	1
1	0	Set (1)	1	0	Set (1)	If CLK = '1', preQ <= D;	
1	1	Invalid	1	1	Toggle	T	Q(Next state)
						0	No change
						1	Toggle

JK: entity JK ff is

port (

J, K, CLK : in std-logic;

Q, Qn : out std-logic);

end JK ff.

architecture behavioral of JK ff is

signal preQ : std-logic := '0';

begin

~~if (CLK = '1') then~~

process (J, K, CLK)

begin

if (CLK = '1') then

if (J = '0' and K = '0') then

preQ <= preQ;

elsif (J = '0' and K = '1') then

preQ <= '0';

elsif (J = '1' and K = '0') then

preQ <= '1';

else

preQ <= not preQ;

end if;

end if;

end process;

Q <= preQ;

Qn <= not preQ;

end behavioral;

Testbench (after port map, before stim-proc)

Date _____
Page _____

clk-process : process

begin

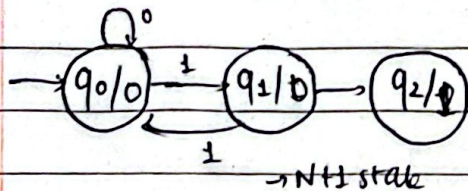
clk <= '0'; wait for 0 ps;

clk <= '1'; wait for 5 ps;

end process;

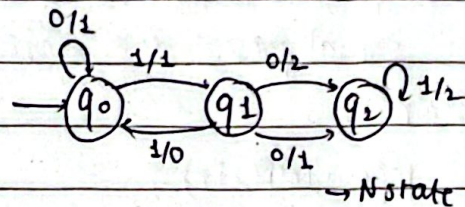
Moore Machine

Output depends only on present state.



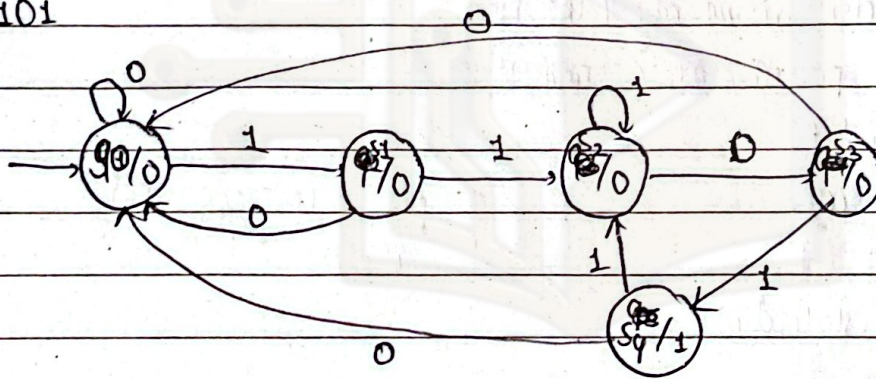
Melay Machine

Output depends on present state and present input.



Sequence detector using Moore Machine

0) Detect 1101



Fsm : 3 blocks

→ process (clk, clr)

→ process (present state, din)

→ process (present-state)

library IEEE;
use IEEE.std-logic-1164.all;

Date _____
Page _____

entity fsm is

port (clk, clr, din : in std-logic;
dout : out std-logic);

end fsm;

Architecture fsm-arch of fsm is

type state is (s0, s1, s2, s3, s4);

signal present-state, next-state : state;

begin

process (clk, clr)

begin

if (clr = '1') then

present-state <= s0;

elsif rising-edge (clk) then

present-state <= next-state;

end if;

end process;

process (present-state, din)

begin

case present-state is

when s0 => if din = '1' then

next-state <= s1;

else

next-state <= s0;

end if;

when s1 => ---

s2, s3, s4 - -

when others => next-state <= s0;

end case;

end process;

process (present-state)

begin

if present-state = s4 then

dout <= '1';

else

dout <= '0';

end if;

end process;

end fsm-arch;

Date _____ Page _____

Operators (Allows to perform various operations on data objects such as signals, variables, constants)

1. Logical operators

(Logical AND, Logical OR, ^LNAND, ^LNOR, L.XOR, L.XNOR, L.NOT)

2. Relational operators

(=, != (Not equal to), <, >, <=, >=)

3. Shift Operators

shift left logical (sll), srl, sla (shift left arithmetic), sra, rol (rotate left), ror

4. Arithmetic operators

+, -, *, /, mod, rem

5. Concatenation operator

& (eg. Vector <= A & B)

6. Miscellaneous operator

? : conditional operator Result <= (A > B) ? A : B

Unit-5 # Wireless communication (communication achieved through electromagnetic waves)

1. Bluetooth

Range : 10-100 m

Speed : upto 2 Mbps (BLE)

Power Efficiency : Optimized for minimal energy usage.

Network type : Supports point-to-point or small network (called piconet)

- Operates in 2.4 GHz ISM band

Types: Bluetooth classic, Bluetooth low Energy (BLE)

Applications: Wearables, Smart Home devices, Audio Devices